

**AN
INTERNSHIP REPORT
On**

CodeVerse

*Submitted
by*

**Akbari Nihar Brijeshbhai
(Enrollment Number: 221430142001)**

Guided By:
Prof. Neha Pujara
An
Internship Report
Submitted to



Gujarat Technological University

In fulfillment for the award of degree of Bachelor of Engineering
in

CSE(AIML)

ACADEMIC YEAR – 2026



NEW L. J. INSTITUTE OF ENGINEERING AND TECHNOLOGY

Pakwan, Behind Rajpath Club Gate to Sindhu Bhavan Road,
Sarkhej - Gandhinagar Hwy, AEC Char Rasta, Ahmedabad, Gujarat 380054



NEW L. J. INSTITUTE OF ENGINEERING AND TECHNOLOGY

CERTIFICATE

This is to certify that the Internship report submitted along with the project entitled Internship in **Grownited Pvt. Ltd.** has been Completed by **Akbari Nihar Brijeshbhai** under my guidance in complete fulfilment for the Bachelor of Engineering in **CSE(AIML)** Branch, 8th Semester of **Gujarat Technological University**, Ahmedabad during the academic year 2026.

Date:

Place: NEW LJJET, Ahmedabad.

Signature and Name of Guide

Prof. Neha Pujara
Assistant Professor (CSE-AIML),
CSE(AIML)/IT Department,
NEW LJJET (143), Ahmedabad.

Signature and Name of H.O.D.

Dr. Gayatri Pandi.
CSE, CSE(AIML), IT Department,
NEW LJJET (143), Ahmedabad.

Signature and Name of Principal

Dr. Gayatri Pandi.
NEW LJJET (143), Ahmedabad

Seal of Institute

INTERNSHIP CERTIFICATE



Date: - 19/04/2026

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Akbari Nihar Brijeshbhai**, a student of New LJ Institute Of Engineering and Technology has successfully completed his/her internship in the field of **JAVA** from 19 /01/2026 to 19/04/2026 under the guidance of **Rahul Kirpekar**.

His/Her internship activities included Project Understanding & Requirement Analysis, Technical Documentation, Learning New Technologies, Development Activities, Testing & Quality Assurance, Soft Skills & Reporting.

During the internship period, he/she was exposed to various processes and was found to be diligent, hardworking, and inquisitive.

We wish him/her all the best in future endeavors.

For, Grownited Private Limited.

Rahul Kirpekar

(Authorised Signature)

301-305, 3rd Floor, Surbhi Complex, Nr. Municipal Market, Chimanlal Girdharlal Road, Navrangpura, Ahmedabad Gujarat- 380009, 7874014621, hr@grownited.com



NEW L. J. INSTITUTE OF ENGINEERING AND TECHNOLOGY

DECLARATION

We hereby declare that the Internship report submitted along with the Internship entitled Project **CodeVerse** submitted in Complete for Bachelor of Engineering in **CSE(AIML)** Branch to **Gujarat Technological University**, Ahmedabad, is a bonafide record of original Internship work Completed by me at **Grownited Pvt. Ltd.** under the supervision of **External Guide Rahul Kirpekar** and that no part of this report has been directly copied from any students' reports or taken from any other source, without providing due reference.

Name of Student

Akbari Nihar Brijeshbhai

Signature of Student

ACKNOWLEDGEMENT

I wish to express my sincere gratitude to my **External guide Rahul Kirpekar** for continuously guiding me at the company and answering all my doubts with patience. I would also like to thank **Dr. Gayatri Pandi (H.O.D. of CSE, CSE(AIML) / IT Department)** for motivating me every time whenever I get confused, I would also like to thank my **Internal Guide Prof. Neha Pujara** for helping me through my internship by giving me the necessary suggestions and advices along with their valuable co-ordination in completing this Internship.

I also thank my parents, friends and all the members of the family for their precious support and encouragement which they had provided in completion of my work. In addition to that, I would also like to mention the company personals who gave me the permission to use and experience the valuable resources required for the Internship.

Thus, in conclusion to the above said, I once again thank the staff members of **Grownited Pvt. Ltd.** for their valuable support in completion of the Internship.

Thank You

Name of Student

Akbari Nihar Brijeshbhai

Enrollment Number

221430142001

Date:

Signature of Student

TABLE OF CONTENTS

CHAPTERS	PAGE NO.
Title Page	i
Certificate Page	ii
Internship Certificate	iii
Declaration	iv
Acknowledgements	v
Table of Contents	vi
Abstract	ix
CHAPTER 1	01
INTRODUCTION OF PROJECT & COMPANY PROFILE	
1.1 Introduction	02
1.1.1 Company Profile	02
1.1.2 Company Products	02
1.1.3 Company Mission and Vision	03
1.2 Introduction of the Project	04
1.2.1 Purpose of the Project	04
1.2.2 Function Requirements	04
1.2.3 Problems in Existing System	04
1.2.4 Main Modules	05
CHAPTER 2 SYSTEM REQUIREMENTS	06
2.1 Hardware & Software Requirements	07
2.1.1 Server-Side Requirements	07
2.1.2 Developer Side Requirements	07
2.1.3 User Side Requirements	07
CHAPTER 3 WORKSHEET REPORT	09
3.1 WORK SHEET REPORT (15 DAYS)	10
CHAPTER 4 FRONT END OF SYSTEM	17

4.1 About Front End	18
4.1.1 about HTML	18
4.1.2 about CSS	20
4.1.3 about JavaScript	21
CHAPTER 5 BACK END OF SYSTEM	24
5.1 About PHP	25
5.2 Why Use PHP	26
5.3 About Back End	27
5.3.1 about MySQL	28
5.3.2 Why use MySQL	30
CHAPTER 6 SYSTEM DESIGN	35
6.1 System Analysis and Design	36
6.2 Systems Development Life Cycle (SDLC)	37
6.2.1 Needs of SDLC	37
6.2.2 SDLC Cycle	38
6.3 Spiral Model	40
6.4 Class Diagram	42
CHAPTER 7 DATA DICTIONARY	44
7.1 Introduction	45
7.2 List of Tables	45
7.2.1: Admin_master	45
7.2.2: User_master	46
7.2.3: Task_master	46
7.2.4: Employee_master	47
7.2.5: Task_Assignment_master	47
7.2.6: Recent_Activity_master	48
7.2.7: Project_master	48
CHAPTER 8 TESTING	49

8.1 Testing Plan	50
8.2 Testing Strategies	52
8.3 Testing Method	55
8.4 Test Case	57
CHAPTER 9 SNAPSHOT OF WEBSITE	60
9.1 Admin Site	61
9.2 Users Site	71
CHAPTER 10 ADVANTAGES	73
10.1 Advantages	74
10.2 Limitations	74
CHAPTER 11 CONCLUSION AND FUTURE ENHANCEMENT	76
11.1 Conclusion	77
11.2 Future Enhancement	77
BIBLIOGRAPHY	79
a. Course Outcome	80
b. Books	81
c. Web Reference	81

CodeVerse

Enrollment No.: 221430142001

Student Name: Akbari Nihar Brijeshbhai

**NEW L. J. INSTITUTE OF ENGINEERING AND TECHNOLOGY
(College Code:143)**

Semester: VIII, CSE(AIML) Department

ABSTRACT

This internship report documents the development of CodeVerse during my internship at Grownited Pvt. Ltd. Grownited Pvt. Ltd provided the work environment, technical guidance, and practical exposure needed to complete the project effectively. CodeVerse is a role-based web application designed to streamline end-to-end hackathon management. The system supports four user roles: Admin, Organizer, Participant, and Judge. Key modules include user authentication, profile management, category and hackathon management, team creation, hackathon applications, judge assignments, scorecards, and payment processing. The platform is built using Spring Boot, JSP, JSTL, MySQL, and Maven, with SMTP-based email support and Cloudinary integration for profile image uploads. The application improves transparency and efficiency by centralizing workflows like registration, application tracking, judging, and result publishing.

Keywords: Hackathon, Team Management, Judge Evaluation, Spring Boot, JSP, MySQL

CHAPTER: 1

INTRODUCTION OF PROJECT & COMPANY PROFILE

1.1. Introduction

1.1.1. Company Profile

1.1.2. Company Products

1.1.3. Company Mission and Vision

1.2. Introduction of the Project

1.2.1. Purpose of the Project

1.2.2. Function Requirements

1.2.3. Problems in Existing System

1.2.4. Main Modules

1.1. Introduction

1.1.1. Company Profile

Grownited Private Limited is a software development company based in Ahmedabad, India. The company focuses on web and mobile application development, enterprise solutions, cloud services, and other digital products that help businesses build scalable technology systems.

1.1.2. Company Products

- The core output is a full-stack web platform for hackathon operations:
- User and role management
- Event and category management
- Team and application workflows
- Judge evaluation and scoring
- Payment and notification support

1.1.3. Company Mission and Vision

Grownited's mission is to deliver secure, scalable, and high-performance technology solutions while encouraging innovation, collaboration, and continuous learning.

1.2. Introduction of the Project

1.2.1. Purpose of the Project

- Automate hackathon registration and participation workflows
- Reduce manual errors and communication gaps
- Provide real-time status visibility for all stakeholders

1.2.2. Function Requirement

- Secure signup/login and session management
- Admin CRUD for users, categories, and hackathons
- Participant team creation and application submission
- Organizer-side application review and judge assignment
- Judge score entry and result generation
- Payment handling for paid hackathons

1.2.3. Problems in Existing System

- Manual spreadsheets and email chains
- Delayed status updates
- Non-standard evaluation process
- Difficulty in audit and reporting

1.2.4. Main Modules

- Authentication Module
- User and Role Management
- Hackathon Management
- Team and Application Module
- Judge Assignment and Scorecard Module
- Payment and Notification Module

CHAPTER: 2

SYSTEM REQUIREMENTS

2.1 Hardware & Software Requirements

2.1.1 Server-Side Requirements

2.1.2 Developer Side Requirements

2.1.3 User Side Requirements

2.1 Hardware & Software Requirements

2.1.1 Server-Side Requirements

- **Hardware Requirement**
 - Intel i5 or higher
 - 8 GB RAM minimum (16 GB recommended)
 - 20 GB free disk space
- **Software Requirement**
 - **Front End:** JSP, JSTL, HTML5, CSS3, JavaScript
 - **Back End:** Java 17, Spring Boot 4.0.2, Spring Data JPA, Hibernate, MySQL 8.x
 - **Server:** Apache Tomcat (embedded via Spring Boot)
 - **Design Tool:** SpringTool Suit
 - **OS:** Windows/Linux/macOS
 - **External Service:** Cloudinary (image hosting), Gmail SMTP (email)

2.1.2 Developer-Side Requirements

- IDE: IntelliJ IDEA / VS Code / Eclipse
- Git
- Postman or browser-based testing
- Chrome/Edge browser

2.1.3 User-Side Requirements

- Modern browser (Chrome, Edge, Firefox)
- Stable internet connection
- JavaScript enabled

CHAPTER: 3

WORK SHEET REPORT

3.1 Work Sheet Report (15 DAYS)

3.1 Work Sheet Report (15 DAYS):

<u>SUGGESTED 15 DAYS WORK SHEET REPORT</u>				
Student Name:		Akbari Nihar Brijeshbhai		
Enrollment No:		221430142001		
Internship/Project Title		CodeVerse		
Tools and Technologies		Java		
Company/ Organization Name		Grownited Pvt. Ltd.		
Student's Activity Details:				
Week Number	Start Date to End Date	Tasks to be assigned	Tasks to be completed	Remarks
1	19/01/2026 To 24/01/2026	- Basics of Java - Logic building & programming - ER diagram	Completed	
2	26/01/2026 To 30/01/2026	- Data dictionary - Use case diagram - System Requirement Specifications (SRS) - UML diagram	Completed	

<u>SUGGESTED 15 DAYS WORK SHEET REPORT</u>				
Student Name:		Akbari Nihar Brijeshbhai		
Enrollment No:		221430142001		
Internship/Project Title		CodeVerse		
Tools and Technologies		Java		
Company/ Organization Name		Grownited Pvt. Ltd.		
Student's Activity Details:				
Week Number	Start Date to End Date	Tasks to be assigned	Tasks to be completed	Remarks
3	02/02/2026 To 06/02/2026	- Table design finalization - Relationships (FK-PK) - Schema Creation in MySQL - Sample Data Insertion	Completed	
4	09/02/2026 To 13/02/2026	- Project Structure setup - Database Connection integration - Controller & Service layer development - Servlet & JSP flow understanding	Completed	

<u>SUGGESTED 15 DAYS WORK SHEET REPORT</u>				
Student Name:		Akbari Nihar Brijeshbhai		
Enrollment No:		221430142001		
Internship/Project Title		CodeVerse		
Tools and Technologies		Java		
Company/ Organization Name		Grownited Pvt. Ltd.		
Student's Activity Details:				
Week Number	Start Date to End Date	Tasks to be assigned	Tasks to be completed	Remarks
5	16/02/2026 To 22/02/2026	- User registration & login flow - Session handling - Input validation - Forgot Password Module	Completed	
6	23/02/2026 To 1/03/2026	- Admin dashboard development - User management module - Category management module - Role-based access control	Completed	

<u>SUGGESTED 15 DAYS WORK SHEET REPORT</u>				
Student Name:		Akbari Nihar Brijeshbhai		
Enrollment No:		221430142001		
Internship/Project Title		CodeVerse		
Tools and Technologies		Java		
Company/ Organization Name		Grownited Pvt. Ltd.		
Student's Activity Details:				
Week Number	Start Date to End Date	Tasks to be assigned	Tasks to be completed	Remarks
7	02/03/2026 To 08/03/2026	- Hackathon creation & listing - Hackathon details page - Registration rules implementation - Status & date handling	Completed	
8	09/03/2026 To 15/03/2026	- Participant dashboard development - My applications page - Apply for hackathon flow - Application status tracking	Completed	

<u>SUGGESTED 15 DAYS WORK SHEET REPORT</u>				
Student Name:		Akbari Nihar Brijeshbhai		
Enrollment No:		221430142001		
Internship/Project Title		CodeVerse		
Tools and Technologies		Java		
Company/ Organization Name		Grownited Pvt. Ltd.		
Student's Activity Details:				
Week Number	Start Date to End Date	Tasks to be assigned	Tasks to be completed	Remarks
9	16/03/2026 To 22/03/2026	- Organizer module development - Judge assignment logic - Evaluation & review flow - Result management	Completed	
10	23/03/2026 To 29/03/2026	- Payment page development - Credit card payment integration - Transaction validation - Payment status handling	Completed	

<u>SUGGESTED 15 DAYS WORK SHEET REPORT</u>				
Student Name:		Akbari Nihar Brijeshbhai		
Enrollment No:		221430142001		
Internship/Project Title		CodeVerse		
Tools and Technologies		Java		
Company/ Organization Name		Grownited Pvt. Ltd.		
Student's Activity Details:				
Week Number	Start Date to End Date	Tasks to be assigned	Tasks to be completed	Remarks
11	02/03/2026 To 08/03/2026	- UI improvements & responsiveness - Email template integration - Error handling & bug fixing - Date formatting updates	Completed	
12	09/03/2026 To 15/03/2026	- Final testing & debugging - Code cleanup & optimization - Documentation completion - Project submission preparation	Completed	

CHAPTER: 4

FRONT END OF SYSTEM

4.1 About Front End

4.1.1 about HTML/JSP

4.1.2 About HTML/JSP

4.1.3 about JavaScript

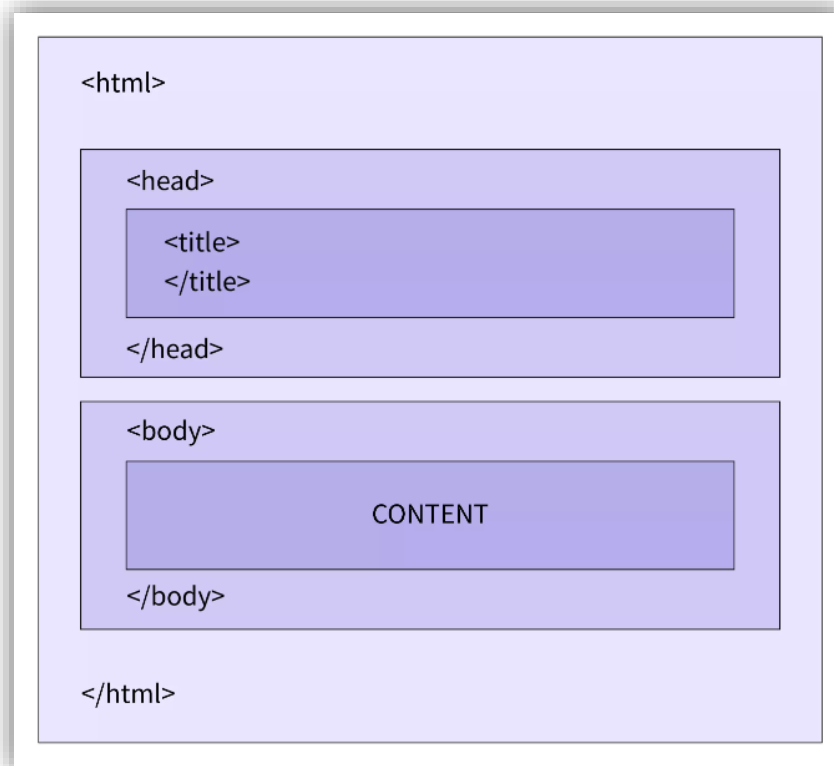
4.2 Major Front End Pages

4.1 About Front End

- The user interface of CodeVerse is built using JSP pages with JSTL tags, HTML, CSS, and JavaScript.
- Role-based views are separated for admin, organizer, participant, and judge.

4.1.1 About HTML/JSP

- JSP view resolution with prefix /WEB-INF/views/ and suffix .jsp
- Reusable page sections and server-side rendered forms
- Dynamic content rendering through JSTL and model attributes
- **HTML Page Structure**



- Any HTML document mainly comprises two parts, the head and the body.

- The Head includes the additional information of the document, apart from the one displayed on the web page.
- It includes the HTML version, the title, and also the metadata.
- And the information that will be displayed on the web page is put inside the Body tag.
- HTML, or Hypertext Markup Language, is the standard language used to create and design web pages.
- It provides the structure and content of a webpage by using a system of markup tags and attributes.

4.1.2 About HTML/JSP

- Custom styles for dashboard pages and forms
- Responsive layout practices for desktop and laptop usage
- Status badges and visual hierarchy for tables/lists

4.1.3 About JavaScript

- Form-level validation and interaction helpers
- - Dynamic field behavior (for example, event type and location rules)
- - Client-side usability enhancements

4.2 Major Frontend Pages

- Login, Signup, Forget Password, Reset Password
- AdminDashboard, ListUser, ListCategory, ListHackathon
- Participant Home, Hackathon Details, MyApplications, MyTeams
- Organizer Applications, JudgeAssignments, Results
- Judge Dashboard and Scorecards

CHAPTER: 5

BACK END OF SYSTEM

5.1 About Backend

5.2 Backend Technologies

5.3 Core Backend Features

5.3.1 about MySQL

5.3.2 How MySQL Works

5.3.3 MySQL Features

5.4 Data Handling

5.5 Important Controller Groups

5.1 About Backend

CodeVerse backend follows layered architecture with Controllers, Services, Repositories, Entities, DTOs, Filters, and Utility classes.

5.2 Backend Technologies

- Java 17
- Spring Boot 4.0.2
- Spring MVC
- Spring Data JPA (Hibernate)
- MySQL Database
- Maven Build

5.3 Core Backend Features

- Session-based authentication with role checks
- BCrypt password hashing
- CSRF protection filter
- Role-based route guarding via AuthFilter
- Mail service for welcome and reset flows
- Payment processing integration

5.4 Database Handling

- Datasource URL configured for MySQL database codeversse
- JPA ddl-auto set to update
- SQL logging enabled for debugging

5.5 Important Controller Groups

- SessionController: authentication and account recovery
- AdminController: dashboard and administration
- Participant controllers: home, teams, applications
- Organizer controllers: applications, judge assignment, profile
- JudgeController: assignments and scorecards
- PaymentController: credit card charge flow

CHAPTER 6

SYSTEM ANALYSIS AND SYSTEM DESIGN

6.1 System Analysis and Design

6.2 Systems Development Life Cycle (SDLC)

6.2.1 Needs of SDLC

6.2.2 SDLC Cycle

6.3 Spiral Model

6.4 Class Diagram

6.1 System Analysis and Design

System Analysis and Design (SAD) is the backbone of software development, guiding the creation of efficient and effective systems to address organizational needs. At its core, SAD is about understanding the current state of affairs within an organization, envisioning a better future, and architecting the pathway to get there.

During the analysis phase, SAD professionals delve deep into the existing systems and processes, meticulously identifying strengths, weaknesses, and opportunities for improvement. Through interviews, surveys, and observation, they gather requirements from stakeholders, ensuring that every voice is heard and every need is accounted for.

With requirements in hand, the design phase begins, where SAD experts transform abstract ideas into concrete plans. They define the system architecture, user interfaces, and data structures, ensuring that the design aligns closely with the gathered requirements and the overarching goals of the organization.

Implementation brings the design to life, as developers translate the blueprints into functional code. Rigorous testing follows, with each component scrutinized to ensure its reliability, security, and performance. User acceptance testing ensures that the system not only meets technical specifications but also satisfies the needs and expectations of its intended users.

Deployment marks the culmination of the SAD process, as the new system is introduced into the organization's ecosystem. With proper training and support, users adapt to the new system, embracing its capabilities and maximizing its potential to drive efficiency and innovation.

But the journey doesn't end there. Maintenance and support are ongoing responsibilities, as SAD professionals monitor the system, address issues, and incorporate updates to keep it running smoothly and meeting the evolving needs of the organization.

Throughout every phase of System Analysis and Design, collaboration, communication, and a relentless focus on the end-users are paramount. By aligning technology with business objectives and user needs, SAD professionals play a vital role in shaping the future success of organizations across industries.

6.2 Systems Development Life Cycle (SDLC)

A software life cycle model (also termed process model) is a pictorial and diagrammatic representation of the software life cycle. A life cycle model represents all the methods required to make a software product transit through its life cycle stages. It also captures the structure in which these methods are to be undertaken.

In other words, a life cycle model maps the various activities performed on a software product from its inception to retirement. Different life cycle models may plan the necessary development activities to phases in different ways. Thus, no element which life cycle model is followed, the essential activities are contained in all life cycle models though the action may be carried out in distinct orders in different life cycle models. During any life cycle stage, more than one activity may also be carried out.

6.2.1 Needs of SDLC

The development team must determine a suitable life cycle model for a particular plan and then observe to it.

Without using an exact life cycle model, the development of a software product would not be in a systematic and disciplined manner. When a team is developing a software product, there must be a clear understanding among team representative about when and what to do. Otherwise, it would point to chaos and project failure. This problem can be defined by using an example. Suppose a software development issue is divided into various parts and the parts are assigned to the team members. From then on, suppose the team representative is allowed the freedom to develop the roles assigned to them in whatever way they like. It is possible that one representative might start writing the code for his part, another might choose to prepare the test documents first, and some other engineer might begin with the design phase of the roles assigned to him. This would be one of the perfect methods for project failure.

A software life cycle model describes entry and exit criteria for each phase. A phase can begin only if its stage-entry criteria have been fulfilled. So without a software

life cycle model, the entry and exit criteria for a stage cannot be recognized. Without software life cycle models, it becomes tough for software project managers to monitor the progress of the project.

6.2.2 SDLC Cycle

SDLC Cycle represents the process of developing software. SDLC framework includes the following steps:



Figure 6.2.1: SDCL Cycle

- The stages of SDLC are as follows:

Stage1: Planning and requirement analysis

- Requirement Analysis is the most important and necessary stage in SDLC.
- The senior members of the team perform it with inputs from all the stakeholders and domain experts or SMEs in the industry.
- Planning for the quality assurance requirements and identifications of the risks associated with the projects is also done at this stage.

Stage2: Defining Requirements

- Once the requirement analysis is done, the next stage is to certainly represent and document the software requirements and get them accepted from the project stakeholders.
- This is accomplished through "SRS"- Software Requirement Specification document which contains all the product requirements to be constructed and developed during the project life cycle.

Stage3: Designing the Software

- The next phase is about to bring down all the knowledge of requirements, analysis, and design of the software project. This phase is the product of the last two, like inputs from the customer and requirement gathering.

Stage4: Developing the project

- In this phase of SDLC, the actual development begins, and the programming is built. The implementation of design begins concerning writing code.
- Developers have to follow the coding guidelines described by their management and programming tools like compilers, interpreters, debuggers, etc. are used to develop and implement the code.

Stage5: Testing

- After the code is generated, it is tested against the requirements to make sure that the products are solving the needs addressed and gathered during the requirements stage.
- During this stage, unit testing, integration testing, system testing, acceptance testing are done.

Stage6: Deployment

- Once the software is certified, and no bugs or errors are stated, then it is deployed.
- Then based on the assessment, the software may be released as it is or with suggested enhancement in the object segment. After the software is deployed, then its maintenance begins.

Stage7: Maintenance

- Once when the client starts using the developed systems, then the real issues come up and requirements to be solved from time to time. This procedure where the care is taken for the developed product is known as maintenance.

6.3 Spiral Model

- The Spiral Model is a Software Development Life Cycle (SDLC) model that provides a systematic and iterative approach to software development. In its diagrammatic representation, looks like a spiral with many loops. The exact number of loops of the spiral is unknown and can vary from project to project. Each loop of the spiral is called a phase of the software development process.
- The exact number of phases needed to develop the product can be varied by the project manager depending upon the project risks.
- As the project manager dynamically determines the number of phases, the project manager has an important role in developing a product using the spiral model.
- It is based on the idea of a spiral, with each iteration of the spiral representing a complete software development cycle, from requirements gathering and analysis to design, implementation, testing, and maintenance.
- The Spiral Model is a risk-driven model, meaning that the focus is on managing risk through multiple iterations of the software development process. It consists of the following phases:
 - Planning
 - o The first phase of the Spiral Model is the planning phase, where the scope of the project is determined and a plan is created for the next iteration of the spiral.
 - Risk Analysis
 - o In the risk analysis phase, the risks associated with the project are identified and evaluated.
 - Engineering
 - o In the engineering phase, the software is developed based on the requirements gathered in the previous iteration.

- Evaluation
 - o In the evaluation phase, the software is evaluated to determine if it meets the customer's requirements and if it is of high quality.
- Planning
 - o The next iteration of the spiral begins with a new planning phase, based on the results of the evaluation.
- The Spiral Model is often used for complex and large software development projects, as it allows for a more flexible and adaptable approach to software development. It is also well-suited to projects with significant uncertainty or high levels of risk.
- The Radius of the spiral at any point represents the expenses(cost) of the project so far, and the angular dimension represents the progress made so far in the current phase.
- Each phase of the Spiral Model is divided into four quadrants as shown in the above figure. The functions of these four quadrants are discussed below:
- Objectives determination and identify alternative solutions:
 - o Requirements are gathered from the customers and the objectives are identified, elaborated, and analyzed at the start of every phase. Then alternative solutions possible for the phase are proposed in this quadrant.
- Identify and resolve Risks:
 - o During the second quadrant, all the possible solutions are evaluated to select the best possible solution. Then the risks associated with that solution are identified and the risks are resolved using the best possible strategy. At the end of this quadrant, the Prototype is built for the best possible solution.
- Develop the next version of the Product:
 - o During the third quadrant, the identified features are developed and verified through testing. At the end of the third quadrant, the next version of the software is available.
- Review and plan for the next Phase:
 - o In the fourth quadrant, the Customers evaluate the so-far developed version of the software. In the end, planning for the next phase is started.

6.4 Class Diagram

The class diagram depicts a static view of an application. It represents the types of objects residing in the system and the relationships between them. A class consists of its objects, and also it may inherit from other classes. A class diagram is used to visualize, describe, document various different aspects of the system, and also construct executable software code.

It shows the attributes, classes, functions, and relationships to give an overview of the software system. It constitutes class names, attributes, and functions in a separate compartment that helps in software development. Since it is a collection of classes, interfaces, associations, collaborations, and constraints, it is termed as a structural diagram.

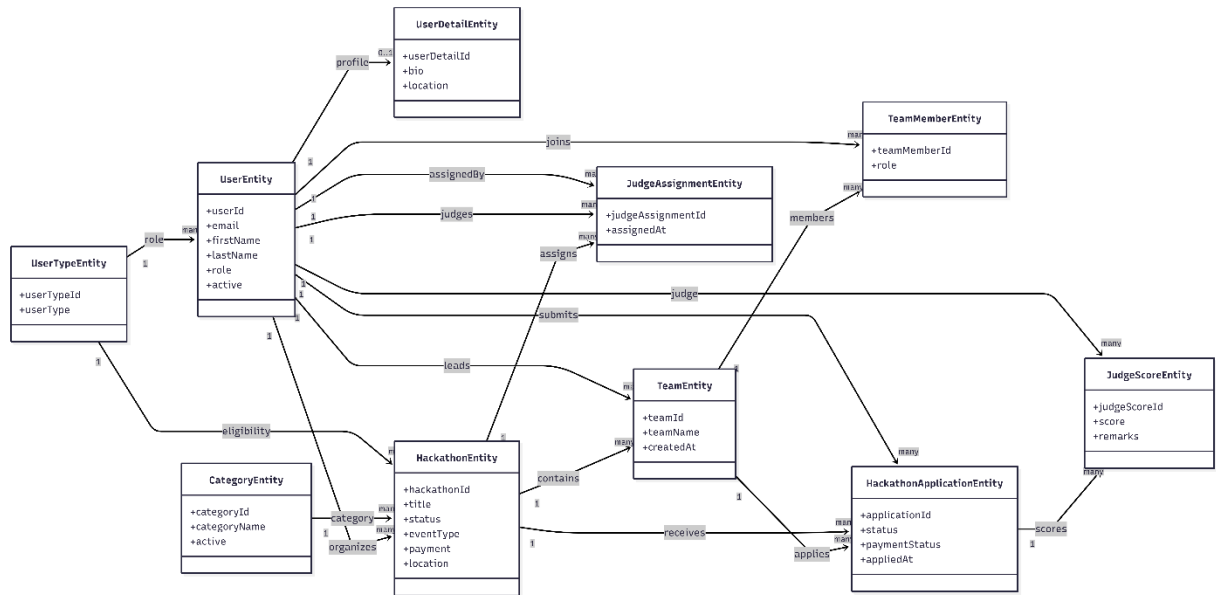
- **Purpose of Class Diagrams**

- o The main purpose of class diagrams is to build a static view of an application. It is the only diagram that is widely used for construction, and it can be mapped with object-oriented languages. It is one of the most popular UML diagrams. Following are the purpose of class diagrams given below:

- It analyses and designs a static view of an application.
- It describes the major responsibilities of a system.
- It is a base for component and deployment diagrams.
- It incorporates forward and reverse engineering.

- **Benefits of Class Diagrams**

- o It can represent the object model for complex systems.
- o It reduces the maintenance time by providing an overview of how an application is structured before coding.
- o It provides a general schematic of an application for better understanding.
- o It represents a detailed chart by highlighting the desired code, which is to be programmed.
- o It is helpful for the stakeholders and the developers.



CHAPTER: 7

DATA DICTIONARY

7.1 Introduction

7.2 List of Tables

7.2.1: user_types

7.2.2: users

7.2.3: user_details

7.2.4: categories

7.2.5: hackathon

7.2.6: teams

7.2.7: team_members

7.2.8: hackathon_applications

7.2.9: judge_assignments

7.2.10: judge_scores

7.1 Introduction

Data Dictionary is an important part of a project or system which contains all definition of elements in the system. In Data Dictionary you will find a list of all elements composing the data flowing through a system.

- The major elements of a system:
 - Data Flow
 - Data Store
 - Processes
- The data dictionary stores all details and description of these elements.
- The data dictionary provides additional information about the system.
- The data dictionary contains the data about data e.g. Student is a data and this student belongs to this course is a description of STUDENT data which is stored in data dictionary.
- Why is Data Dictionary important?
 - To manage the details in large system.
 - To communicate a common meaning for all system elements.
 - To document the features of system.
 - To determine where system changes should made.
 - To locate errors and omissions in the system.

7.2 List of Tables:

- **Table 7.2.1: user_type**
 - **Description: Stores user types of system (Admin/Organizer/Participant/Judge).**

[Table 7.2.1: user_types]

Field Name	Data type (size)	Constraint	Description
user_type_id	BIGINT	Primary key, Auto Increment	Unique role id
user_type	Varchar(30)	Not Null, Unique	Role Name (ADMIN/ORGANIZER/PARTICIPANT/JUDE)

- **Table 7.2.2: users**

- **Description:** Stores login and basic profile details

[Table 7.2.2: users]

Field Name	Data type (size)	Constraint	Description
user_id	BIGINT	Primary key, Auto Increment	Stores the unique identifier of the user.
first_name	Varchar(100)	Not Null	User's first name
last_name	Varchar(100)	Not Null	User's last name
email	Varchar(150)	Not Null, Unique	User's email
password	Varchar(255)	Not Null	Encrypted password
role	VARCHAR(30)	NOT NULL	Role label
profile_pic_url	VARCHAR(500)	NULL	Profile image url
active	BOOLEAN	NOT NULL	Account Status
otp	VARCHAR(10)	NULL	• OTP for password reset
user_type_id	BIGINT	FK -> user types.user type id	Role reference

- **Table 7.2.3: user_details**

- **Description:** Stores extended user profile information

[Table 7.2.3: users_details]

Field Name	Data type (size)	Constraint	Description
user_detail_id	BIGINT	Primary key, Auto Increment	PK, Auto Increment
user_id	BIGINT	FK -> users.user_id, UNIQUE	Owner user
bio	VARCHAR(1000)	NULL	Short profile bio
location	VARCHAR(150)	NULL	User location
linkedin_url	VARCHAR(255)	NULL	LinkedIn Profile link
role	VARCHAR(30)	NOT NULL	Role label
profile_pic_url	VARCHAR(500)	NULL	Profile image url

- **Table 7.2.4: catagories**

- **Description:** Stores hackathon categories.

[Table 7.2.4: catagories]

Field Name	Data type (size)	Constraint	Description
category_id	BIGINT	PK, Auto Increment	Unique category id
category_nam	VARCHAR(100)	NOT NULL, UNIQUE	Category name

active	BOOLEAN	NOT NULL	Category status
--------	---------	----------	-----------------

- **Table 7.2.5: hackathon**
 - **Description: Stores hackathon categories.**

[Table 7.2.5: hackathon]

Field Name	Data type (size)	Constraint	Description
hackathon_id	BIGINT	PK, Auto Increment	Unique hackathon id
title	VARCHAR(200)	NOT NULL	Hackathon Title
description	TEXT	NOT NULL	Event description
status	VARCHAR(30)	NOT NULL	UPCOMING/ONGOING/COMPLETED
event_type	VARCHAR(30)	NOT NULL	ONLINE/OFFLINE/HYBRID
payment	VARCHAR(30)	NOT NULL	FREE/PAID
location	VARCHAR(200)	NULL	Event location
registration_start_date	DATE	NOT NULL	Registration start
registration_end_date	DATE	NOT NULL	Registration end
min_team_size	INT	NOT NULL	Minimum team members
max_team_size	INT	NOT NULL	Maximum team members
user_id	BIGINT	FK -> users.user_id	Organizer id
category_id	BIGINT	FK -> categories.category_id	Category id

- **Table 7.2.6: teams**
 - **Description:** Stores team records for hackathons.

[Table 7.2.6: teams]

Field Name	Data type (size)	Constraint	Description
team_id	BIGINT	PK, Auto Increment	Unique team id
team_name	VARCHAR(150)	NOT NULL	Team Name
created_at	DATETIME	NOT NULL	Team creation timestamp
leader_user_id	BIGINT	FK -> users.user_id	Team leader

- **Table 7.2.7: team_members**
 - **Description:** Stores members belonging to teams.

[Table 7.2.7: team_members]

Field Name	Data type (size)	Constraint	Description
team_member_id	BIGINT	PK, Auto Increment	Unique member row id
team_id	VARCHAR(150)	FK -> teams.team_id	Team reference

member_user_id	BIGINT	FK -> users.user_id	Member user
role	VARCHAR(30)	NOT NULL	LEADER/MEMBER

- **Table 7.2.8: hackathon_applications**
 - **Description:** Stores application submissions for hackathons.

[Table 7.2.8: team_members]

Field Name	Data type (size)	Constraint	Description
application_id	BIGINT	PK, Auto Increment	Unique application id
team_id	BIGINT	FK -> teams.team_id	Team reference
participant_user_id	BIGINT	FK -> users.user_id	Applicant user
status	VARCHAR(30)	NOT NULL	PENDING/APPROVED/REJECTED/ etc
payment_status	VARCHAR(30)	NOT NULL	WAIVED/PENDING/PAID
applied_at	DATETIME	NOT NULL	Application timestamp
submission_url	VARCHAR(500)	NULL	Project submission URL
frontend_github_link	VARCHAR(500)	NULL	Frontend repo URL
backend_github_link	VARCHAR(500)	NULL	Backend repo URL

submission_description	TEXT	NULL	Submission details
------------------------	------	------	--------------------

- **Table 7.2.9: judge_assignments**
 - **Description: Stores team records for hackathons.**

[Table 7.2.9: judge_assignments]

Field Name	Data type (size)	Constraint	Description
judge_assignment_id	BIGINT	PK, Auto Increment	Unique assignment id
hackathon_id	BIGINT	FK -> hackathon.hackathon_id	Hackathon reference
judge_user_id	BIGINT	FK -> users.user_id	Assigned judge
assigned_by_user_id	BIGINT	FK -> users.user_id	Organizer/Admin who assigned
assigned_at	DATETIME	NOT NULL	Assignment timestamp

- **Table 7.2.10: judge_scores**
 - **Description:** Stores scorecards given by judges.

[Table 7.2.10: judge_scores]

Field Name	Data type (size)	Constraint	Description
judge_score_id	BIGINT	PK, Auto Increment	Unique score id
application_id	BIGINT	BIGINT FK -> hackathon_applications.application_id	Application reference
judge_id	BIGINT	FK -> users.user_id	Judge Reference
score	DECIMAL(5,2)	NOT NULL	Numeric score
remarks	VARCHAR(1000)	NULL	Judge comments

CHAPTER:8

TESTING

8.1 Testing Plan

8.2 Testing Strategies

8.3 Testing Method

8.4 Test Case

8.1 Testing Plan

Great! Let's outline a detailed testing plan. Since you haven't specified the exact product or system you're testing, I'll provide a generic outline that you can adapt to your specific needs. Here's a structured testing plan:

- **Introduction:**
 - Provide an overview of the testing plan.
 - Include the objectives, scope, and key stakeholders.
- **Purpose:**
 - Define the purpose of testing.
 - Clarify what aspects of the product/system will be tested.
- **Scope:**
 - Describe the boundaries of the testing.
 - Identify the features, functionalities, or components included in the scope.
 - Mention any excluded items or functionalities.
- **Testing Goals:**
 - List the specific goals or objectives of the testing.
 - These goals should align with the overall project objectives and stakeholder requirements.
- **Testing Strategy:**
 - Outline the overall testing approach.
 - Specify the types of testing to be conducted (e.g., functional testing, usability testing, performance testing, security testing).
 - Describe any testing methodologies or frameworks to be followed (e.g., Agile, Waterfall, DevOps).
- **Test Deliverables:**
 - List the documents, reports, or artifacts that will be produced during testing.
 - Include items such as test plans, test cases, test scripts, and test reports.
- **Testing Environment:**
 - Specify the hardware, software, and network configurations required for testing.
 - Detail any test environment setup or configurations needed.

- **Test Cases:**
 - Develop detailed test cases for each identified test scenario.
 - Include preconditions, steps to execute, expected results, and post-conditions.
 - Organize test cases by priority and complexity.
- **Test Execution:**
 - Describe how test cases will be executed.
 - Assign responsibilities for test execution.
 - Define the criteria for determining when testing is complete.
- **Defect Management:**
 - Explain how defects/issues will be logged, tracked, and managed.
 - Define severity and priority levels for defects.
 - Outline the defect resolution process.
- **Risks and Mitigation:**
 - Identify potential risks associated with testing.
 - Propose mitigation strategies for each identified risk.
- **Schedule:**
 - Create a testing timeline or schedule.
 - Allocate time for test preparation, execution, and reporting.
 - Consider any dependencies or constraints that may impact the schedule.
- **Roles and Responsibilities:**
 - Define the roles and responsibilities of team members involved in testing.
 - Specify who is responsible for test planning, execution, documentation, and communication.
- **Communication Plan:**
 - Outline how communication will be managed throughout the testing process.
 - Identify key stakeholders and their communication preferences.
 - Specify the frequency and format of status updates and reports.
- **Review and Approval:**
 - Establish a process for reviewing and approving the testing plan.
 - Identify the stakeholders responsible for review and approval.
- **Documentation:**

- Document any changes or updates to the testing plan.
 - Maintain a version history for reference.
- **Conclusion:**
 - Summarize the key points of the testing plan.
 - Confirm alignment with project goals and stakeholder expectations.
- **Appendices:**
 - Include any supplementary information or reference materials relevant to the testing plan.

Once you've drafted your testing plan, be sure to review it with your team and stakeholders to ensure clarity, alignment, and agreement before proceeding with testing activities.

8.2 Testing Strategies

Testing strategies are crucial for ensuring that the testing process is efficient, thorough, and effective. Here's a detailed breakdown of various testing strategies:

- **Unit Testing:**
 - Definition: This strategy involves testing individual components or units of code in isolation.
 - Focus: It verifies the correctness of each unit's behavior and functionality.
 - Approach: Developers typically write unit tests using testing frameworks like JUnit (for Java), NUnit (for .NET), or pytest (for Python).
 - Benefits: Unit testing helps identify defects early in the development cycle and promotes code modularity and reusability.
- **Integration Testing:**
 - Definition: Integration testing verifies the interactions between different units or modules within the system.

- o Focus: It ensures that integrated components work together as expected.
- o Approach: Integration tests can be conducted using techniques such as top-down, bottom-up, or a combination of both.
- o Benefits: Integration testing detects interface defects and integration issues early, reducing the risk of major failures during system testing.

- **System Testing:**

- o Definition: System testing evaluates the entire system as a whole, including all integrated components.
- o Focus: It validates the system against its functional and non-functional requirements.
- o Approach: System testing can include functional testing, usability testing, performance testing, security testing, and more.
- o Benefits: System testing ensures that the software meets user expectations and performs reliably in its intended environment.

- **Acceptance Testing:**

- o Definition: Acceptance testing determines whether the system meets the acceptance criteria defined by stakeholders.
- o Focus: It validates the system from the end-user's perspective and assesses its readiness for deployment.
- o Approach: Acceptance testing can be performed using techniques such as alpha testing, beta testing, or user acceptance testing (UAT).

- o Benefits: Acceptance testing ensures that the software satisfies business requirements and aligns with user needs.

- **Regression Testing:**

- o Definition: Regression testing verifies that recent code changes have not adversely affected existing functionality.

- o Focus: It ensures that previously tested features still work correctly after modifications or enhancements.

- o Approach: Automated regression testing tools can help efficiently re-run existing test cases to detect regressions.

- o Benefits: Regression testing helps maintain software quality and prevents the introduction of unintended defects during development.

- **Exploratory Testing:**

- o Definition: Exploratory testing involves simultaneous learning, test design, and test execution.

- o Focus: Testers explore the application dynamically, uncovering defects and potential use cases.

- o Approach: Testers rely on their domain knowledge and intuition to design and execute tests on-the-fly.

- o Benefits: Exploratory testing is effective for finding defects that may not be identified through scripted tests and encourages creative and flexible testing approaches.

- **Risk-Based Testing:**

- o Definition: Risk-based testing prioritizes testing efforts based on the likelihood and impact of potential failures.
 - o Focus: It focuses testing resources on high-risk areas of the system.
 - o Approach: Risk analysis techniques, such as Failure Mode and Effects Analysis (FMEA), help identify critical areas for testing.
 - o Benefits: Risk-based testing optimizes testing efforts by directing resources where they are most needed, reducing the likelihood of high-impact failures.
- **Load Testing:**
 - o Definition: Load testing evaluates the system's performance under expected and peak load conditions.
 - o Focus: It assesses how the system handles concurrent user activity and data processing.
 - o Approach: Load testing tools simulate high-volume traffic to measure system response times, throughput, and scalability.
 - o Benefits: Load testing identifies performance bottlenecks, helps optimize resource utilization, and ensures the system can handle expected workloads.
- **Security Testing:**
 - o Definition: Security testing identifies vulnerabilities and weaknesses in the system's security controls.
 - o Focus: It evaluates the system's ability to protect data, prevent unauthorized access, and resist attacks.

- o Approach: Security testing includes techniques such as penetration testing, vulnerability scanning, and code reviews.
- o Benefits: Security testing helps mitigate security risks, safeguard sensitive information, and maintain the integrity of the system.
- **Usability Testing:**
 - o Definition: Usability testing assesses the system's user interface and user experience (UI/UX).
 - o Focus: It evaluates how easily users can interact with the system and accomplish their tasks.
 - o Approach: Usability testing involves observing real users as they interact with the system and collecting feedback through surveys or interviews.
 - o Benefits: Usability testing identifies usability issues, improves user satisfaction, and enhances the overall user experience.

Each of these testing strategies plays a critical role in ensuring the quality, reliability, and performance of software systems. The selection and combination of testing strategies depend on factors such as project requirements, constraints, and risks. It's essential to tailor the testing approach to the specific needs of the project and continuously adapt it throughout the software development lifecycle.

8.3 Testing Method

Testing methods refer to the specific approaches or techniques used to conduct testing activities. These methods help ensure thorough coverage, accuracy, and effectiveness in validating the software's functionality, performance, and other quality attributes. Here are some common testing methods:

- **Black Box Testing:**

- o Description: In black box testing, the tester does not have access to the internal code or logic of the software. Instead, tests are designed based on the software's specifications and requirements.
- o Approach: Test cases are created to validate inputs, outputs, and system behavior without knowledge of the internal implementation.
- o Benefits: Black box testing focuses on the software's external behavior, allowing testers to identify issues from a user's perspective.

- **White Box Testing:**

- o Description: White box testing, also known as glass box or clear box testing, involves examining the internal structure and logic of the software.
- o Approach: Test cases are designed based on an understanding of the code, control flow, and data flow within the software.
- o Benefits: White box testing provides insights into code coverage, control flow, and error handling, helping identify issues related to code execution and logic.

- **Gray Box Testing:**

- o Description: Gray box testing combines elements of both black box and white box testing. Testers have partial knowledge of the internal workings of the software.
- o Approach: Test cases are designed based on a limited understanding of the internal code or structure, allowing testers to target specific areas of interest.
- o Benefits: Gray box testing leverages both external and internal perspectives,

providing a balance between test coverage and depth.

- **Manual Testing:**

- o Description: Manual testing involves human testers executing test cases manually without the use of automation tools.

- o Approach: Testers interact with the software interface, input data, and observe outputs to validate its behavior.

- o Benefits: Manual testing allows for exploratory testing, subjective evaluation of user experience, and validation of complex scenarios that may be challenging to automate.

- **Automated Testing:**

- o Description: Automated testing involves the use of software tools to execute pre-scripted tests, compare actual outcomes with expected results, and generate test reports.

- o Approach: Test scripts are created to automate repetitive test scenarios, regression tests, and performance tests.

- o Benefits: Automated testing improves efficiency, repeatability, and coverage of testing activities, especially for large-scale projects and continuous integration pipelines.

- **Static Testing:**

- o Description: Static testing focuses on reviewing code, requirements, and documentation without executing the software.

- o Approach: Techniques such as code reviews, walkthroughs, and inspections are

used to identify defects and improve quality early in the development lifecycle.

- o Benefits: Static testing helps uncover defects at an early stage, reducing the cost and effort of fixing issues later in the development process.

- **Dynamic Testing:**

- o Description: Dynamic testing involves executing the software and observing its behavior to validate its functionality, performance, and other quality attributes.

- o Approach: Techniques such as unit testing, integration testing, system testing, and performance testing fall under dynamic testing.

- o Benefits: Dynamic testing verifies the software's behavior in a real-world environment, uncovering defects that may not be apparent through static analysis alone.

- **Model-Based Testing:**

- o Description: Model-based testing involves creating abstract models of the system's behavior and using them to generate test cases automatically.

- o Approach: Models, such as finite state machines or UML diagrams, are analyzed to derive test scenarios and expected outcomes.

- o Benefits: Model-based testing improves test coverage, reduces manual effort in test case design, and ensures consistency between requirements and tests.

These testing methods can be combined and adapted based on project requirements, constraints, and goals. Effective testing often involves a combination of multiple methods to achieve comprehensive coverage and ensure software quality.

8.4 Test Case

A test case is a detailed set of conditions or steps that are designed to verify the functionality,

behavior, or performance of a software application or system. Test cases are used to validate

whether the software meets specific requirements and functions correctly under various scenarios. Here's a breakdown of what a test case typically includes:

- **Test Case ID:** A unique identifier for the test case, usually in the form of a combination of letters and numbers.
- **Test Case Title/Name:** A descriptive title or name that summarizes the purpose or objective of the test case.
- **Description:** A brief description or summary of what the test case is intended to achieve.
- **Preconditions:** Any prerequisites or conditions that must be met before executing the test case, such as system configurations, data setup, or user permissions.
- **Test Steps:** A sequence of step-by-step instructions outlining the actions to be performed to execute the test case. Each step should be clear, concise, and unambiguous.
- **Input Data:** The specific data values, parameters, or inputs that will be used during the test execution.
- **Expected Results:** The expected outcomes or behaviors that the system should exhibit when the test case is executed successfully. This includes both observable results and system responses.
- **Actual Results:** The actual outcomes observed during test execution. Testers record the results obtained after executing the test steps.
- **Pass/Fail Criteria:** Criteria used to determine whether the test case passes or fails based on a comparison between the actual results and the expected results.
- **Priority:** The priority level assigned to the test case, indicating its relative importance or urgency in the testing process.

- **Severity:** The severity level assigned to the test case, indicating the impact of a failure on the system or its users. Common severity levels include critical, major, minor, and trivial.
- **Test Environment:** Information about the test environment, including hardware, software, configurations, and dependencies required for test execution.
- **Test Data:** Any additional test data or conditions relevant to the test case, such as boundary values, edge cases, or negative scenarios.
- **Attachments/References:** Any supporting documents, screenshots, logs, or references that provide additional context or details for the test case.
- **Notes/Comments:** Any additional notes, comments, or observations relevant to the test case, such as insights, assumptions, or recommendations.

By documenting test cases in a structured and standardized format, teams can ensure consistent testing practices, facilitate test execution, and maintain comprehensive test coverage throughout the software development lifecycle. Additionally, well-written test cases serve as valuable documentation for future reference, regression testing, and knowledge transfer within the team.

CHAPTER: 9

SNAPSHOT OF WEBSITE

9.1 Admin Side

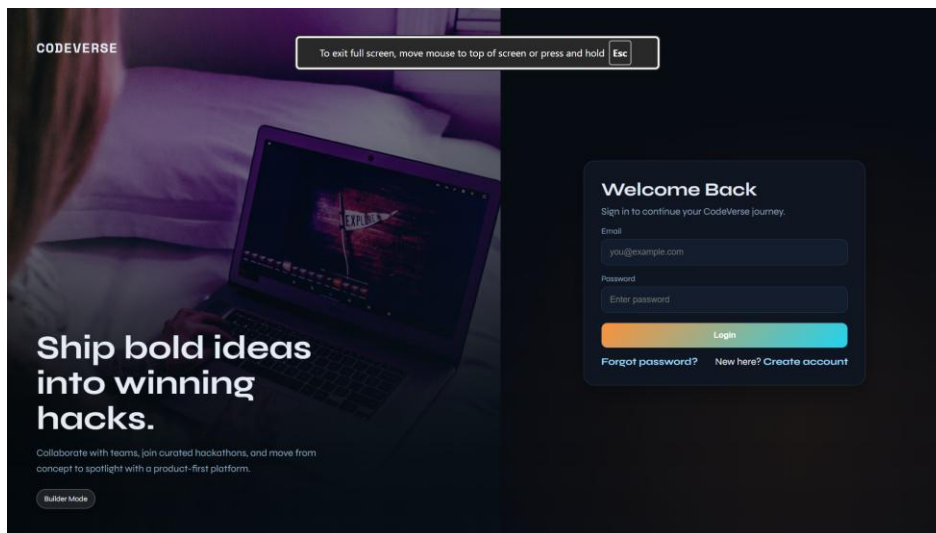
9.2 Participant Side

9.3 Organizer Side

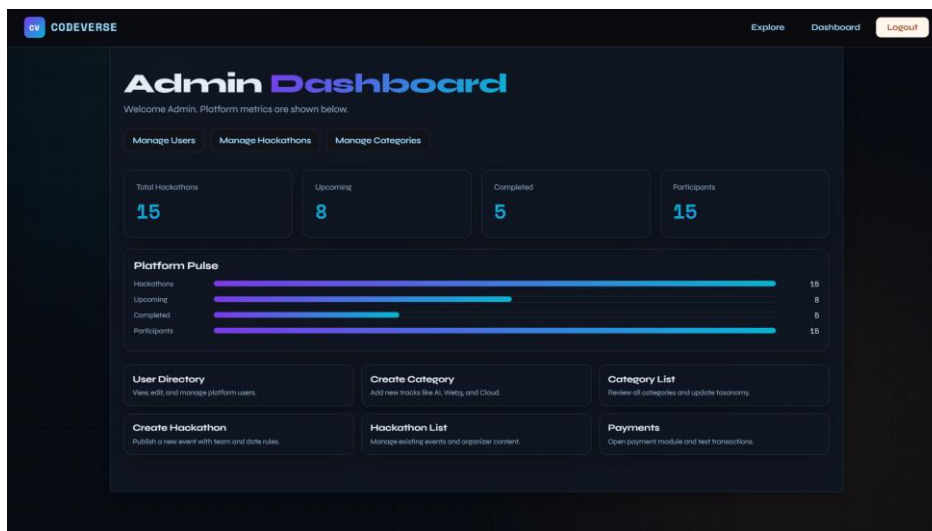
9.4 Judge Side

9.1 Admin Side

- Admin login:

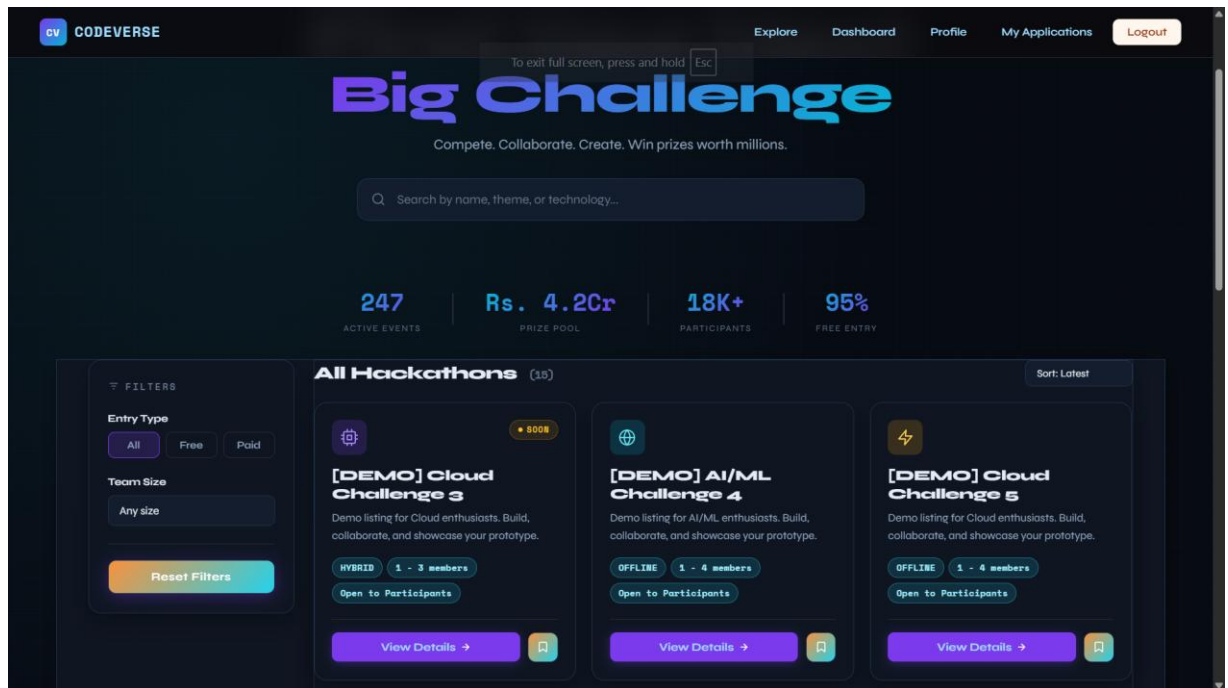


- Dashboard:

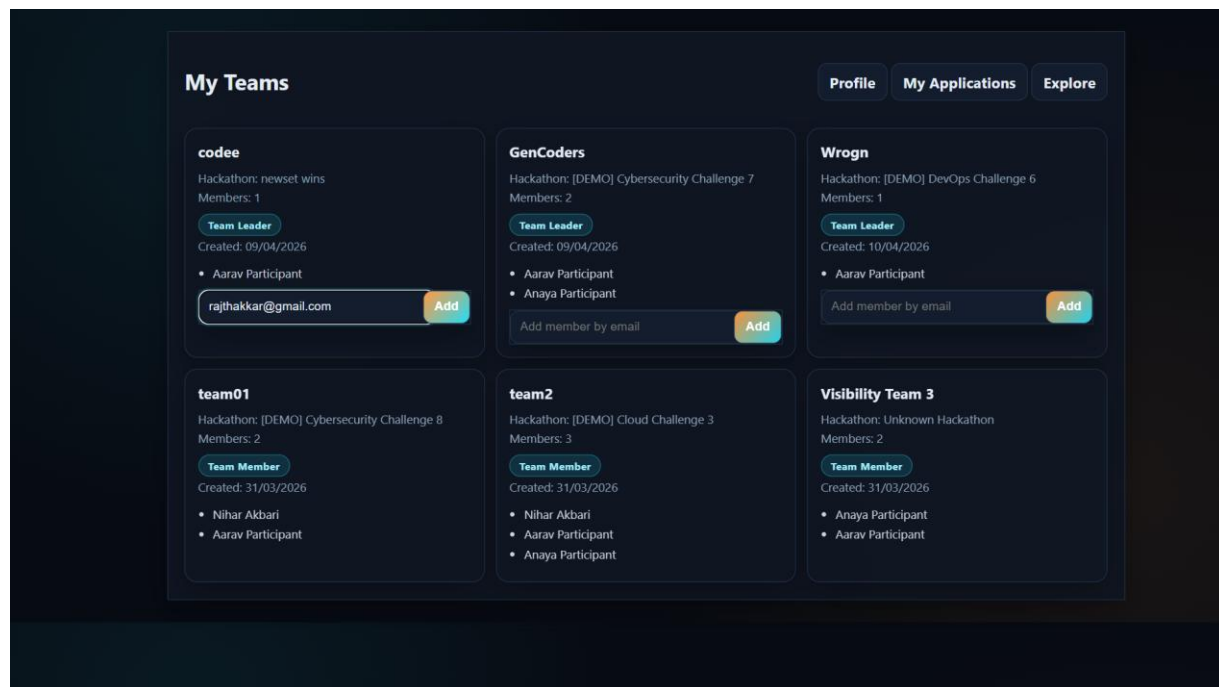


9.2 Participant Side

- Explore Session



- Apply with Team



- **Submission**

My Applications
Track decisions, payment status, and submit your work from one place.

TOTAL **3** ACTION **Submit or update**

Profile My Teams Explore Hackathons

HACKATHON	TEAM	STATUS	PAYMENT	APPLIED ON	SUBMISSION
newset wins Application #8	codee	APPLIED	PENDING	09/04/2026	Submission Saved: Update Submitting work does not change application status; organizers update status later. Current Demo Link https://drive.com Frontend repository URL (optional) Backend repository URL (optional) samples Submit And Save
[DEMO] Cybersecurity Challenge 7 Application #9	GenCoders	APPLIED	PENDING	09/04/2026	Open Submission Form
[DEMO] DevOps Challenge 6 Application #11	Wrogn	APPLIED	PENDING	10/04/2026	Open Submission Form

9.3 Organizer Side

- **Create Hackathon**

Create Hackathon
Configure challenge details, team rules, and registration timeline.

Hackathon List Dashboard

Hackathon Title

Status **-- Select Status --** Event Type **-- Select Event Type --**

Payment **-- Select Payment Type --** Minimum Team Size

Maximum Team Size

Location

Registration Start Date **dd-mm-yyyy** Registration End Date **dd-mm-yyyy**

Description

Cancel Save Hackathon

- **Assign Judge**

Judge Assignments Dashboard

Hackathon: [DEMO] Organizer Evaluation Event

Judge: -- Select Judge --

Assign Judge

HACKATHON

[DEMO] Organizer Evaluation Event

Riya Judge (judge1@codeverse.dev)

Vivek Judge (judge2@codeverse.dev)

Ishita Judge (judge3@codeverse.dev)

Manav Judge (judge4@codeverse.dev)

9.4 Judge Side:

- **Judge Scoring**

Judge Scorecards My Assignments Dashboard

Hackathon: New 2 Load Scorecards

Application #10

Participant: Nihar Akbari

Status: APPLIED

Payment: WAIVED

Submission Details:

hope you would like this

Project Link ↗

Idea & Innovation (0-25)

Design & UX (0-25)

Execution & Code (0-25)

Pitch & Presentation (0-25)

Total Score

Not Scored Yet

Strengths, weaknesses, innovation, execution...

Save Score

You can resubmit to update your previous score.

CHAPTER: 10

ADVANTAGES

10.1 Advantages

10.2 Limitations

10.1 Advantages

- Centralized role-based hackathon management improves control, security, and accountability across Admin, Organizer, Participant, and Judge workflows.
- Faster and more transparent application tracking helps users view real-time status updates, reducing confusion and repeated manual follow-ups.
- Better coordination between organizers and judges enables smoother assignment, evaluation, and communication throughout the event lifecycle.
- A structured result and scorecard process standardizes judging criteria, making outcomes fairer and easier to audit.
- Database-backed workflows improve data consistency, reduce duplicate records, and ensure reliable reporting for teams, applications, and results.
- The modular and scalable architecture supports future enhancements such as analytics, APIs, mobile integration, and automation features.

10.2 Limitations

CodeVerse offer significant benefits, they also have some limitations:

- Manual testing effort is still significant, especially for multi-role flows and edge cases across registration, application, and evaluation modules.
- Advanced analytics, smart notifications, and deeper insight dashboards are currently limited and need further development.
- Snapshot/documentation sections require final curated images and presentation formatting before final academic submission.
- Third-party integrations (email, cloud storage, payment gateway) can introduce external dependency risks and occasional service delays.
- As user volume grows, performance tuning and monitoring will be necessary to maintain smooth experience under higher load.

CHAPTER: 11

CONCLUSION AND FUTURE ENHANCEMENT

11.1 Conclusion

11.2 Future Enhancement

11.1 Conclusion

The CodeVerse Hackathon Management Platform offers an efficient solution for organizing hackathon events end-to-end. Developed during an internship at Grownited Pvt Ltd, it streamlines workflows for administrators, organizers, judges, and participants through role-based access and automation.

Utilizing Spring Boot, MySQL, JSP, and integrations with Cloudinary, Authorize.net, and Gmail SMTP, the system analyses user data to manage registrations, submissions, evaluations, and results. This reduces manual effort, enhances accuracy via secure handling and database normalization, and ensures consistency in event coordination.

The project showcases how layered architecture and modular development can simplify complex processes, boosting productivity in tech education. Overall, CodeVerse transforms hackathon management into a faster, more reliable system, fostering innovation and setting the stage for future advancements in the coding community.

11.2 Future Enhancements

- Add real-time notifications using WebSocket
- Add advanced analytics dashboards
- Add file upload versioning for submissions
- Integrate plagiarism checks for code submissions
- Add REST API and mobile app support
- Add automated test suite and CI pipeline

BIBLIOGRAPHY

a. Course Outcome

b. Books

c. Web Reference

a. Course Outcome

Sr. No.	CO statement	Marks % weightage	Chapter No.
CO-1	Undertake problem identification, formulation and solution	20%	CHAPTER 1 INTRODUCTION
CO-2	Design engineering solutions to complex problems utilising a systematic approach and team work	30%	CHAPTER 2 SYSTEM REQUIREMENTS CHAPTER 3 DAILY TASK
CO-3	Communicate with engineers and the community at large in written and oral forms	20%	CHAPTER 6 SYSTEM DESIGN CHAPTER 10 ADVANTAGES CHAPTER 11 CONCLUSION AND FUTURE ENHANCEMENT
CO-4	Demonstrate the knowledge and understanding of engineering and management principle and apply it to assigned project	30%	CHAPTER 4 FRONT END OF SYSTEM CHAPTER 5 BACK END OF SYSTEM CHAPTER 6 SYSTEM DESIGN CHAPTER 7 DATA DICTIONARY CHAPTER 8 TESTING

b. Books:

- **Fundamental of Software Engineering**
 - Author: Rajib Mall
- **The Joy of PHP Programming: A Beginner's Guide to Programming Interactive Web Applications with PHP and MySQL**
 - Author: Alan Forbes
- **Head First PHP & MySQL**
 - Author: Lynn Beighley & Michael Morrison
- **SQL Server Database**
 - Author: Mr.Gaurav Desani

c. Web Reference:

- **W3Schools. "CSS Tutorial." [Online].**
 - Available: <https://www.w3schools.com/css/default.asp>.. [Accessed: February 1, 2024].
- **W3Schools. "JavaScript Tutorial." [Online].**
 - Available: <https://www.w3schools.com/js/default.asp>. [Accessed: February 7, 2024].
- **Javatpoint. "PHP Tutorial."**
 - [Online] Available: <https://www.javatpoint.com/php-tutorial> [Accessed: February 21, 2024].
- **Free Code Camp. "Learn to code — for free." [Online].**
 - Available: <https://www.freecodecamp.org/> [Accessed: March 15, 2024].
- **Stack Overflow. "Where Developers Learn, Share, & Build Careers." [Online].**
 - Available: <https://stackoverflow.com/> [Accessed: April 2, 2024]